



FINAL SYSTEM, INTEGRATION & DEPLOYMENT

AI-Powered Driver Wellness & Safety Monitoring System

Five trained models unified behind one standardized contract, fused into a live 0–100 wellness score, and deployed as a hosted real-time driver-monitoring application.



Kushagra



Shiwani



Shubham



Sohini

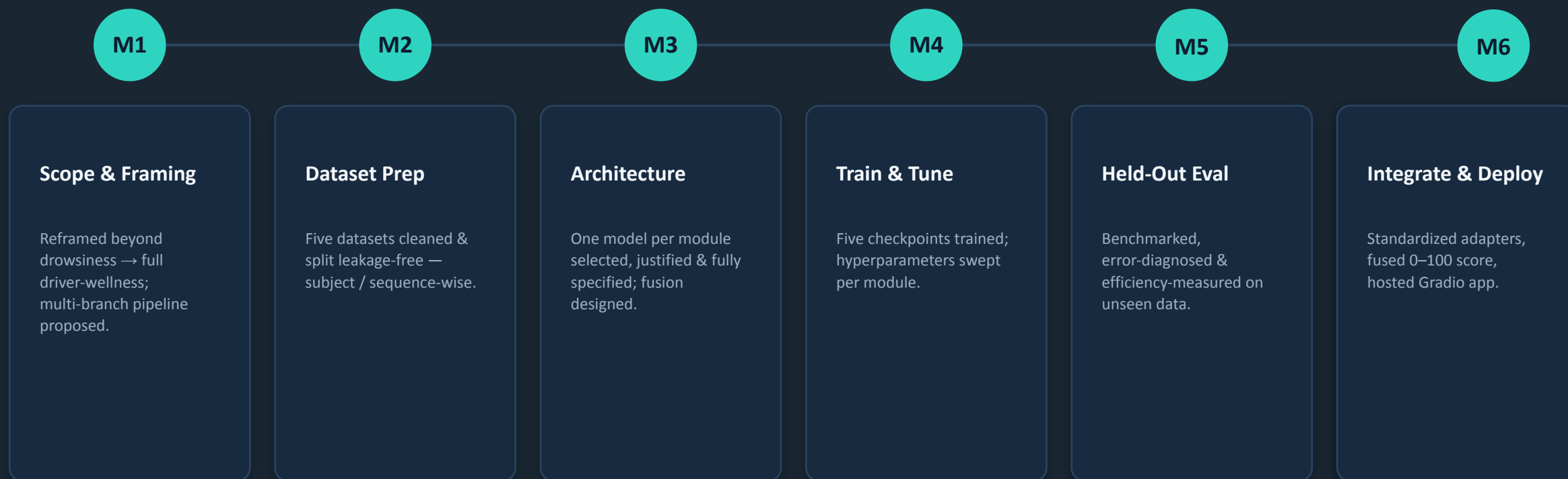


Ravina

BSDA4001 · DS/AI Lab Course · Final Project Presentation

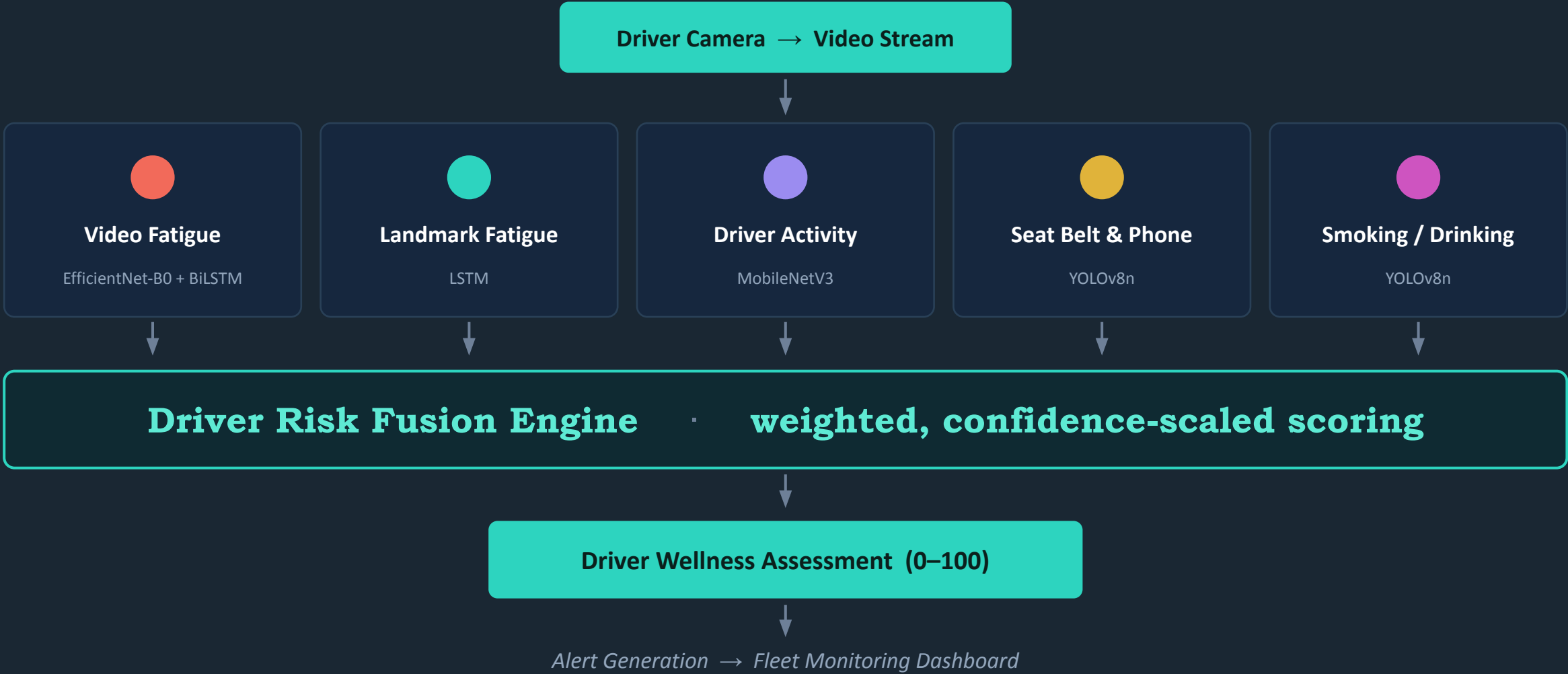
Six Milestones, One System

From a revised scope to a fully fused, deployed real-time wellness engine — each milestone built directly on the last.



You are here → Milestone 6 turns five evaluated models into one live, deployed safety signal.

Five Models, One Wellness Score



Video-Based Fatigue Detection

EfficientNet-B0 + BiLSTM · UTA-RLDD · Subject-Disjoint Evaluation — Milestone 6 Final Presentation



Kushagra Barodekar
Video Fatigue — Module Owner

- Built EfficientNet-B0 + BiLSTM temporal pipeline on UTA-RLDD with subject-disjoint splits.
- Locked one-time test evaluation and integrated adapter into Risk Fusion Engine.
- Down-weighted module in fusion (~33.6% test acc ≈ random 3-class baseline).

Where This Module Sits in the System



YOU ARE HERE
↓ risk-weighted fusion

Risk Fusion Engine → Driver Wellness Score



Problem Statement, Dataset & Architecture

The Problem

Detect fatigue from short in-cabin video while generalizing to unseen drivers. Session labels (Safe / Caution / High_Risk) may not appear in every 3.2s clip.

The Solution

EfficientNet-B0 frame features + BiLSTM temporal modeling on 16-frame windows (~5 FPS). Production adapter maps to Low / Medium / High Risk for fusion.

Architecture Pipeline — Current Integration Build



UTA-RLDD: Train 46,085 · Val 7,042 · Test 7,117 · 60 subjects · subject-disjoint splits · 7.16M params.

M6 DESIGN PRINCIPLE

Don't trust aggregate accuracy alone — inspect per-class recall and cross-driver transfer.

Experiments & Final Results

E1–E5 model selection → E5 (EfficientNet-B0 + BiLSTM) locked for full-data training

Experiment Journey (E1–E5)

E1 — Partial EfficientNet unfreeze

Acc 50.07% · High_Risk recall → 0%

E2 — Weighted CrossEntropy

Minimal gain · imbalance not root cause

E3 — Landmark feature fusion

Macro F1 49.80% · High_Risk recall → 0%

E4 — Landmarks + unfreeze

Best mini Macro F1 51.12% · recall still 0%

E5 — EfficientNet-B0 + BiLSTM

Stable full-data path · non-zero class coverage · **SELECTED**

Selection rule for safety-critical fatigue

Higher accuracy alone was not enough if High_Risk recall collapsed to 0%. E5 was chosen for stable full-data behavior and non-zero class coverage.

CONFUSION MATRIX EVIDENCE

Validation showed all classes were learnable; locked test set exposed stronger cross-driver variation.

Validation Split

A \ P	Safe	Ctn	Risk
Safe	37.0%	27.4%	35.6%
Ctn	40.2%	32.4%	27.4%
Risk	19.9%	13.7%	66.4%

Test Split

A \ P	Safe	Ctn	Risk
Safe	39.8%	2.8%	57.4%
Ctn	32.1%	33.9%	34.0%
Risk	51.8%	22.4%	25.8%

ROWS: ACTUAL | COLUMNS: PREDICTED

Controlled Fusion Impact

- RISK_FUSION_WEIGHTS: video_fatigue 0.20 → 0.10
- VIDEO_FATIGUE_TRUST_FACTOR: 0.50 on R_i (severity × confidence)
- Module remains enabled — complementary signal, not dominant.
- Integration rule: Low Risk = 0 · unavailable modules excluded.

Integration, Limitations & Deployment

Known Limitations

Cross-Driver Generalization Gap

Locked test accuracy $\approx 33.6\%$ (near 3-class random baseline). Safe $\leftrightarrow$ High_Risk confusion dominates on short windows.

Session Label vs Window Mismatch

3.2s windows may not show visible fatigue even when the full session is labeled High_Risk. Research prototype — not safety-certified.

Deployment Footprint

EfficientNet-B0 + BiLSTM ($\sim 7.16\text{M}$ params), ~ 19.7 ms/window — fast enough for real-time fusion alongside four other models.

Integration Status — Deployment Ready

- VideoFatigueAdapter registered in DriverWellnessModuleManager.
- Fusion consumes PredictionResult only — no model-specific fusion code.
- Primary: GitHub $\rightarrow$ Lightning AI $\rightarrow$ Gradio demo.
- Fallback: Kaggle GPU $\rightarrow$ run_kaggle_gradio_fallback.py $\rightarrow$ share=True URL.

Key Takeaway

Not a standalone decision-maker. Adds temporal fatigue context while fusion down-weights its impact. Functional pipeline · evidence-based selection · honest evaluation · safer fusion behavior.

NEXT PERFORMANCE LEVERS

Longer windows • Multi-modal fusion • More subjects • Per-class recall tuning • Night/IR data

Landmark-Based Fatigue Detection

EAR · MAR · Head Pose — Temporal LSTM Classification | Milestone 6 Final Presentation



Shiwani Tiwari

Landmark-Based Fatigue Detection — Module Owner

- Designed and trained the EAR/MAR/head-pose LSTM fatigue classifier end to end since Milestone 2.
- Owned the fix for the TA-flagged action-vs-fatigue-state discrepancy — built and empirically validated the rule that converts predictions into a clinical fatigue state.
- Ran and reported every M4/M5 experiment for this module: label correction, hyperparameter sweep, threshold sensitivity analysis.
- M6: integrating the module into the live five-model pipeline and validating its output in the Risk Fusion Engine.

Where This Module Sits in the System

Video Fatigue

Landmark Fatigue

YOU ARE HERE

Driver Activity

Seat Belt & Phone

Smoking & Drinking

↓ *weighted fusion*

Risk Fusion Engine → Driver Wellness Score

Problem Statement, Pipeline & Starting Baseline

The Problem

Detect driver fatigue in real time from facial video using only geometric signals — no clinical sensors. YawDD only labels physical actions (Normal/Talking/Yawning), not fatigue severity — Milestone 3 trained an action classifier but reported it as if it predicted fatigue state directly.

TA-flagged gap: no fatigue label exists in this dataset to validate against — closed via a documented, empirically-validated rule-based bridge.

Milestone 3 Baseline (what this module had to beat)

40.8%

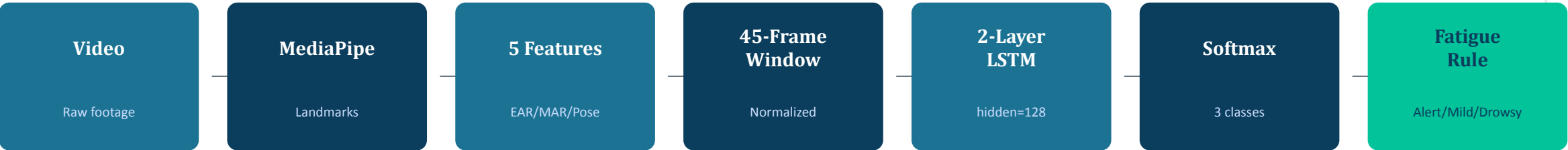
Accuracy

38.4%

Macro F1

4-class model, incl. an invalid 13-video "Talking_Yawning" compound class; every window inherited its whole video's label.

Pipeline Architecture



5 features (EAR/MAR/Pitch/Yaw/Roll) — already human-interpretable, no CNN backbone needed. A "Yawning" prediction can be checked directly against the raw MAR trace.

Experiments & Final Results

Label correction → hyperparameter sweep → threshold sensitivity analysis

Three Experiments

1. Label Structure Fix

Dropped invalid Talking_Yawning class (4.06%). Per-window MAR-peak relabeling for Yawning. → Biggest single driver: +23 pts accuracy.

2. Hyperparameter Sweep

Window size 20/30/45, hidden units, LR, batch. Selected: window=45, hidden=128, lr=0.0005, batch=32. <2-pt spread across sweep.

3. Threshold Sensitivity (M5)

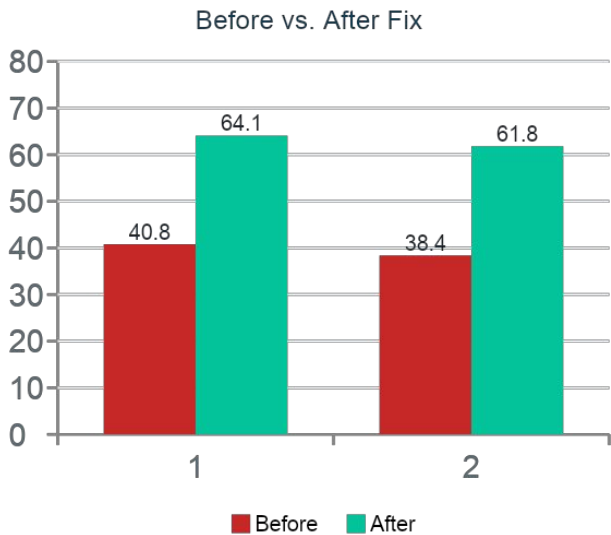
Swept 5/10/15/20% yawn-proportion cutoff against sensitivity/specificity. Shifted 15%→5%: 94.4% drowsiness sensitivity.

```
Class-weighted loss: nn.CrossEntropyLoss([1.477, 1.520, 3.029])
```

Final Test Results (1,400 windows)

Class	P	R	F1
Normal	0.63	0.78	0.70
Talking	0.67	0.52	0.58
Yawning	0.60	0.55	0.58
Macro	0.63	0.61	0.62

Accuracy 64.1% | Macro F1 61.8%



Confusion Matrix

	N	T	Y
N	504	109	35
T	266	306	22
Y	26	45	87

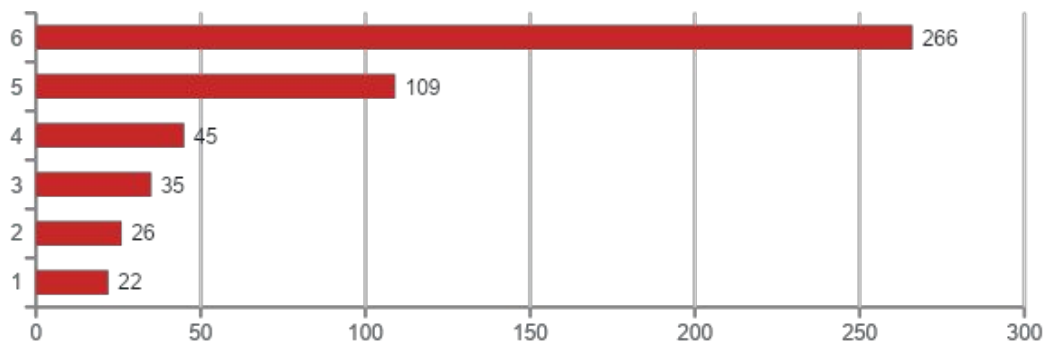
Threshold Sensitivity (M5)

Thr.	Sens.	Spec.
5%	94.4%	91.2%
10%	88.9%	94.1%
15%	83.3%	94.1%
20%	66.7%	97.1%

5% selected — 15% was strictly dominated by 10% (same false alarms, more missed drowsy drivers).

Error Analysis, Limitations & Milestone 6 Status

Error Breakdown (503 misclassified, 35.9%)



Root Cause & Known Limitations

- Talking→Normal (52.9% of errors) is a label-granularity issue, not model capacity — the per-window fix that worked for Yawning made Talking worse when tried.
- Startup latency: needs 45 consecutive valid frames (~1.5-2s) before first prediction.
- Small threshold-validation set (~54 videos) — a clear trend, not a statistically precise optimum.
- Non-deterministic hyperparameter sweep (no fixed seed) — trend is stable, exact numbers vary run to run.

Milestone 6 — Integration Status

Ready:

- Lightweight LSTM, no CNN backbone
- `predict_from_buffer()` streaming-ready
- Checkpoint wired into `wellness_core.py`
- Threshold empirically validated (5%)

Outstanding:

- Wire streaming buffer into live pipeline (joint w/ Kushagra)
- Formal latency/throughput benchmarking
- Confirm weighting in recalibrated Risk Fusion Engine

Key Takeaway

Fixing what the model was trained to predict (label structure) beat every hyperparameter tuning effort combined — +23 points vs. under 2 points from tuning. Every threshold and formula in this module is now documented and empirically justified, closing every TA-flagged gap from M4.

Driver Activity Classification

MobileNetV3-Large · 5-Class Distraction Detection · Edge Deployment — Milestones 1–6 Summary

Where This Module Sits in the System



Shubham Tiwari

Driver Activity Detection — Module Owner

Trained MobileNetV3-Large from scratch to classify five in-cabin driver activities in real time.

Owned the full M1–M6 arc: literature review, dataset curation, architecture selection, training & tuning, evaluation, and Gradio/Hugging Face deployment.

Selected MobileNetV3-Large over ResNet50 / EfficientNet-B0 for the best accuracy-per-millisecond trade-off on edge hardware.

93.14%

Test Accuracy

93.00%

Precision

93.00%

Recall

12.5 ms

MobileNetV3 inference

Problem, Literature Review & Architecture Selection

The Problem

Classify 5 in-cabin driver activities (safe driving, texting, talking, turning, other) from a single RGB frame. M1's literature review found rule-based cues (e.g. EAR thresholds) are fast but fragile across lighting and pose — real dashcam footage needs a model that learns visual patterns directly.

The Approach

M2 curated & cleaned the training data (removed 2,969 duplicate + 10 corrupted images via hashing/OpenCV checks). M3 then benchmarked three CNN backbones on a candidate subset and selected MobileNetV3-Large — trading a few accuracy points for a 6× smaller, edge-deployable model.

Architecture Selection — Backbone Comparison (M3 Baseline, 20% candidate set)

Model	Accuracy	Precision	Recall	F1-Score
MobileNetV3 ✓	89.35%	89.74%	89.35%	89.39%
ResNet50	92.13%	92.32%	92.13%	92.14%
EfficientNet-B0	93.98%	94.08%	93.98%	93.98%

✓ selected — best accuracy-per-millisecond trade-off for edge deployment

Pipeline Architecture — M2–M4 Build



M1–M3 DESIGN PRINCIPLE Highest offline accuracy isn't the goal — highest accuracy per millisecond, on real edge hardware, is.

Training, Hyperparameter Tuning & Final Results

Three Core Improvements (M4)

1 Learning-Rate & Optimizer Tuning

Lowering LR from 0.001→0.0003 and switching Adam→AdamW raised validation accuracy from 87.04% to 91.76%.

2 Regularization Stack

Augmentation + label smoothing ($\alpha=0.1$) + weight decay (0.01) cut the train/val overfitting gap from 12.22% to 0.82%.

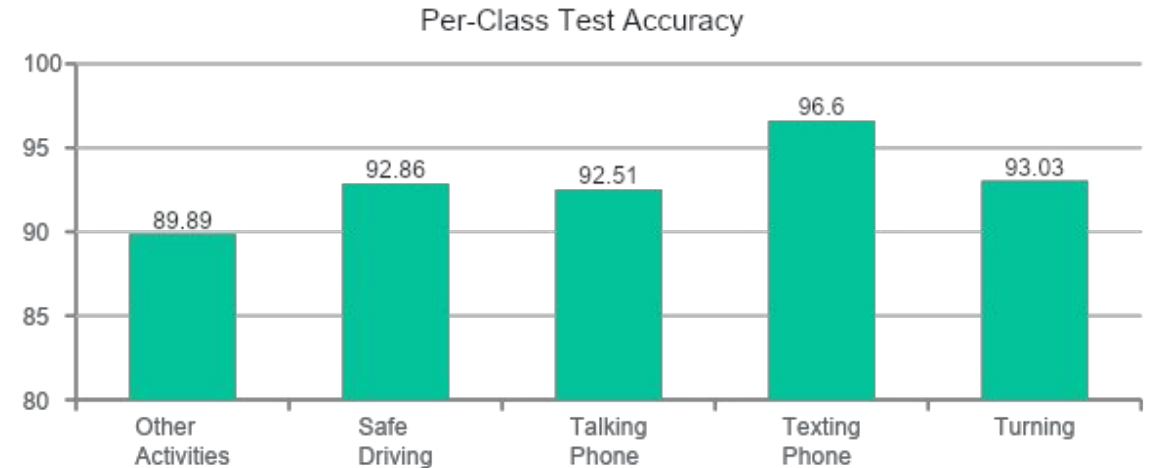
3 Cosine Scheduling & Early Stopping

CosineAnnealingWarmRestarts + early stopping (patience=12) reached 94.41% best val accuracy at epoch 25/80 in 11.77 min on a T4 GPU.

Regularization Impact (Validation Accuracy)

Stage	Val Acc	Gap
No Regularization	85.12%	12.22%
+ Augmentation	88.76%	5.80%
+ Label Smoothing	90.45%	1.89%
All Techniques	94.41%	0.82%

Test Results — Per-Class Accuracy (M5 Benchmark)

**93.14%**

Test Accuracy

93.00%

Macro F1-Score

4.2M

Parameters

12.5 ms

Inference (80 FPS)

Integration, Deployment & Limitations

Root Cause & Known Limitations

Single-frame input: no temporal context, so “turning” is confused with “safe driving” in 19 cases — steering motion only shows across frames.

“Other activities” catch-all: lowest per-class accuracy (89.89%) from high intra-class visual variance.

Deployment blocked: Hugging Face Space runtime inference blocked by a ZeroGPU quota limit at M6 time of writing.

Integration Status — Deployment Ready

DriverActivityAdapter registered in the shared DriverWellnessModuleManager — standardized PredictionResult contract, no model-specific glue code.

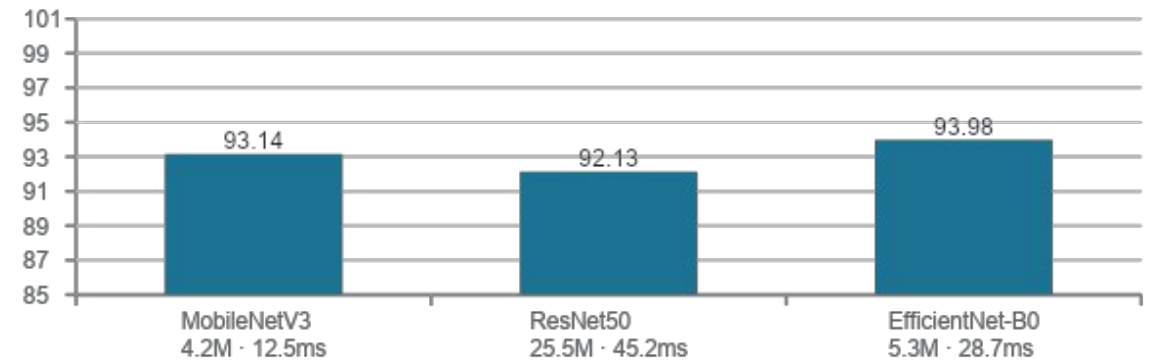
Gradio “Cockpit HUD” interface: recorded-video + live-webcam tabs with an SVG risk gauge.

Accuracy & speed preserved: 93.14% accuracy and 12.5 ms / 80 FPS unchanged from standalone to integrated system.

Key Takeaway

Across M1–M6, the Driver Activity module moved from a literature-driven design choice to a fully trained, tuned, and integrated MobileNetV3-Large classifier — 93.14% test accuracy at 12.5 ms/frame — wired into the Risk Fusion Engine (20% weight) and shipped behind a Gradio demo, with Hugging Face deployment the one open item.

Why MobileNetV3? — Backbone Comparison (M5)



NEXT PERFORMANCE LEVERS Temporal modeling (LSTM/TCN) • INT8 quantization • Knowledge distillation • WebRTC live streaming • Mobile deployment

Seat Belt & Phone Usage Detection

YOLOv8n · Class-Agnostic NMS · Temporal Consensus — Milestone 6 Final Presentation



Sohini Sarkar

Seat Belt & Phone Detection — Module Owner

- Designed and trained the YOLOv8n detection pipeline for seatbelt compliance and phone-distraction monitoring.
- Owned M6 deployment fixes: Gamma 1.4 lighting correction and confidence-floor tuning to resolve the overlapping ‘seesaw’ effect.
- Decoupled per-class confidence floors and tightened temporal buffers.

Where This Module Sits in the System



YOU ARE HERE

↓ risk-weighted fusion

Risk Fusion Engine → Driver Wellness Score

0.953

mAP@50

0.937

Precision

0.907

Recall

~10 ms

YOLOv8n inference

Problem Statement, Pipeline & Baseline

The Problem

Detect seatbelt compliance and phone-distraction in highly variable in-cabin lighting. Raw per-frame detection isn't enough – real footage flickers and boxes overlap.

The Solution

A streaming pipeline that corrects lighting, applies per-class confidence floors, and requires temporal consensus before alerting – turning a noisy per-frame detector into a stable state machine.

Pipeline Architecture — Current Integration Build



Lightweight YOLOv8n backbone (~3.2M params, ~10ms/frame). Since M6, live-video testing surfaced a new failure mode – a phone-holding arm being flagged as a seatbelt; so, the latest integration build turns Class-Agnostic NMS off and instead relies on per-class confidence floors (Phone ≥ 0.15 , Seatbelt ≥ 0.20) plus a 0.5s temporal-consensus window (30% activation / 20% release hysteresis) to stabilize detections.

M6 DESIGN PRINCIPLE

Don't alert on a single noisy frame – correct → filter → stabilize → classify.

Experiments & Final Results

M6 confidence-floor tuning → temporal-buffer tuning → post-M6 NMS reversal

Three Core M6 Enhancements

1. Confidence Floor Tuning

Raised the Seatbelt confidence floor to eliminate low-confidence false positives from earphone-wire micro-artifacts.

2. Buffer Flushing (Temporal)

Replaced a fixed grace-frame counter with a 0.5s sliding-window consensus filter (30% activation / 20% release), so transient flicker is flushed immediately.

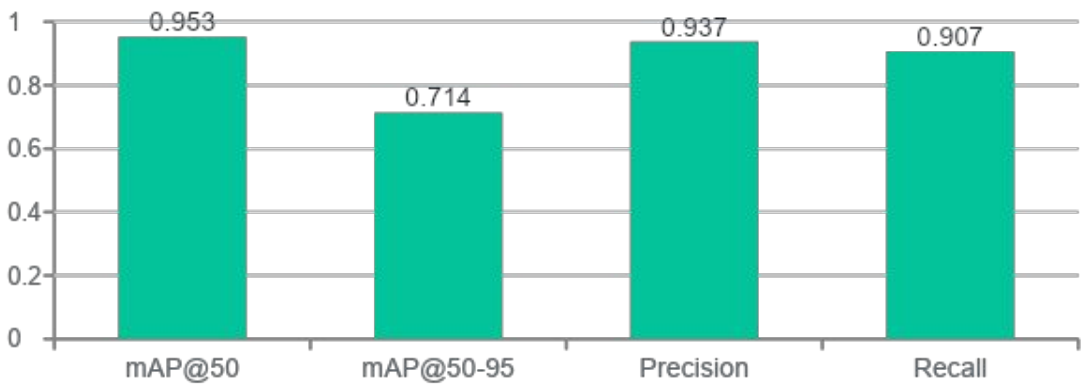
3. The ‘Seesaw’ Fix – Reversed Post-M6

Class-Agnostic NMS solved M6's overlapping-box problem, but later testing found it could suppress genuine Phone boxes near a Seatbelt box. The current build turns it off and leans on per-class floors + augmented inference instead.

Why per-class floors instead of one global value?

Phone needs an ultra-low floor (0.15) to pierce shadows; Seatbelt needs a strict floor (0.20) to reject sunlight glare – one shared threshold under-served both classes.

Test Results — M6 Benchmark (Notebook Test Set)



BENCHMARK NOTE • 0.953 mAP@50 / 0.937 precision are the M6 notebook-test results; the post-M6 NMS reversal was a streaming-robustness decision.

Current Integration Config

Parameter	Value
Base YOLO conf / IoU	0.10 / 0.40
Phone confidence floor	≥ 0.15
Seatbelt confidence floor	≥ 0.20
Temporal window	0.5s (ON 30% / OFF 20%)
Class-Agnostic NMS	Off (reversed post-M6)
Augmented inference	On

Error Analysis, Limitations & Integration Status

Root Cause & Known Limitations

Lighting Glare & Domain Shift

Extreme sunlight washes out seatbelt straps. Lack of heavy nighttime / IR imagery remains an active system limitation.

Notebook-Test Benchmark

The 0.953 mAP score is a notebook benchmark, prioritizing integrated streaming robustness over a strictly held-out static test score.

Integration Status – Deployment Ready

- Registered in the shared DriverWellnessModuleManager alongside the other four adapters – no model-specific logic outside the adapter.
- Kaggle GPU fallback (share=True) configured to bypass Lightning.ai VRAM limits for the live demo.
- Risk Fusion severity audited: ‘Seatbelt Only’ and ‘Safe Driving’ map to severity 0.0, so safe driving never mathematically penalizes the driver.

Deployment Footprint

Lightweight YOLOv8n (~3.2M params), ~10ms/frame – fast enough for real-time in-cabin inference alongside the other four models in the fused pipeline.

Key Takeaway

Every M6 fix – the confidence floors, the sliding-window buffer, and the (since reversed) Class-Agnostic NMS. Test result reached 0.953 mAP@50, and the Risk Fusion severity audit ensures safe driving is never mathematically penalized. The M6 work shifted the detector from raw frame predictions toward a more stable, class-aware streaming decision process.

NEXT PERFORMANCE LEVERS

Night/IR data • Adaptive exposure • Threshold calibration • Track-level temporal voting



A clean, class-balanced dataset

One Roboflow source, name-harmonized to two classes — smoking & drinking — integrity-validated, deduplicated, and balanced with real images rather than heavy augmentation.

8,333

Raw images downloaded

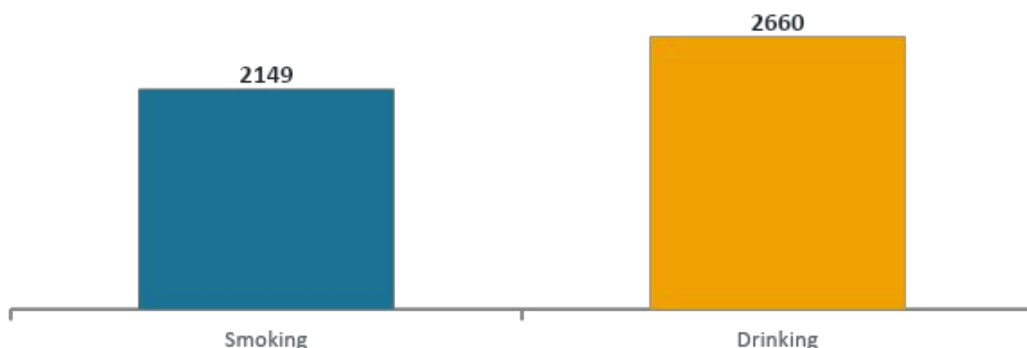
3,704

Images after dedup + balance

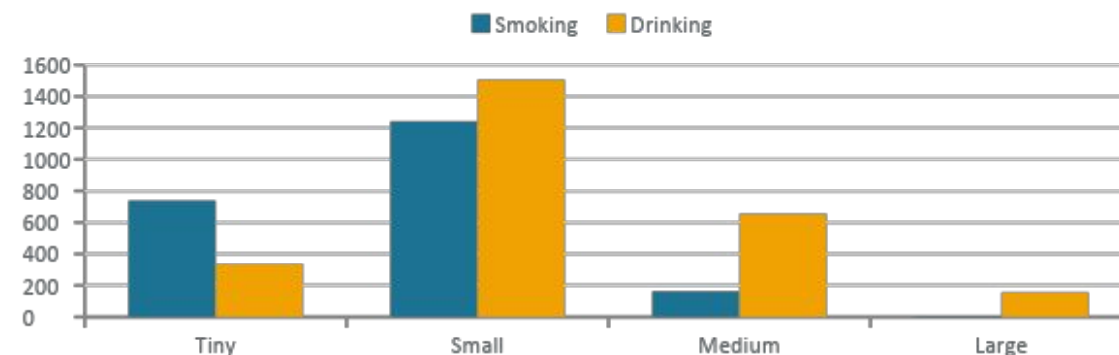
1.24×

Instance imbalance ratio

Class balance — instances (4,809 total)



Object size buckets — per class



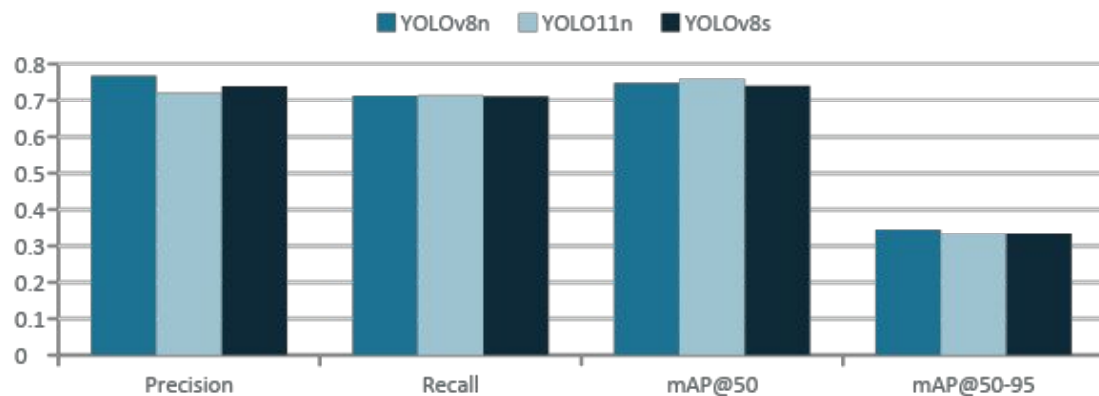
Real-image balance, not augmentation: Earlier pipelines inflated the minority class ~17× (overfitting risk); new version caps the majority class with real images and keeps augmentation light. Non-target classes (eating, mobile) are dropped so they can't leak into a target label, and perceptual dedup removes 365 near-duplicates before the split — so val/test stay leak-free. ~22.4% of boxes are tiny (mostly cigarettes)· split 2,963 / 370 / 371.



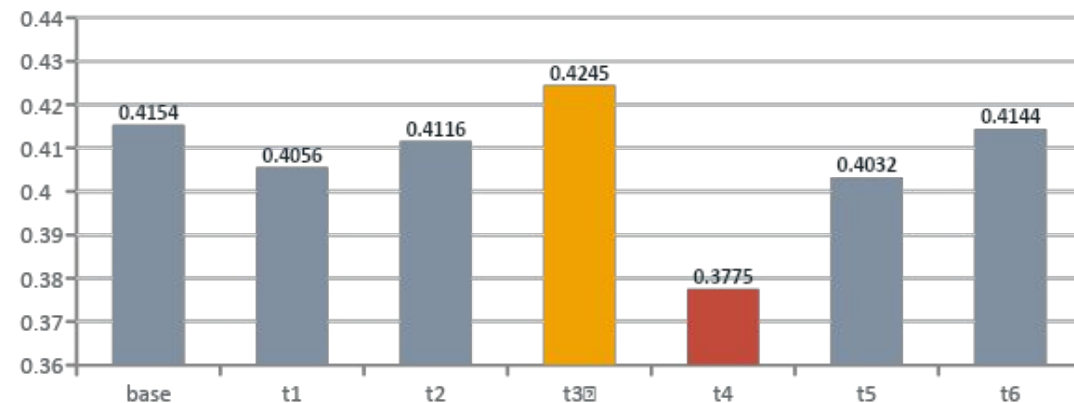
YOLOv8n selected, then tuned

Three anchor-free YOLO detectors were compared under identical conditions on a representative 3,110-image subset; the best-localizing nano won, then a guided sweep produced the 80-epoch final model.

Baseline metrics — default config, 30 ep, val fold



Sweep — val mAP@50-95 per trial (t3 wins)



WINNING CONFIG

t3_lr001_adamw

lr0 = 0.001 · AdamW · cosine LR · mosaic closed last 10 epochs · retrained from COCO for 80 epochs · 3.01 M params · 8.1 GFLOPs · ≈6.9 ms/frame (~145 FPS, T4).

TUNING GAINS — WINNER (t3) vs BASELINE, VAL FOLD

Precision
+2.5%
0.856 → 0.878

Recall
-2.7%
0.789 → 0.767

mAP@50
+0.5%
0.841 → 0.845

mAP@50-95
+2.2%
0.415 → 0.425



Honest takeaway: Capacity didn't help — YOLOv8s (~3× params) failed to beat the nanos. Trial margins are narrow (~0.01 mAP@50-95), so the model isn't brittle to reasonable choices; AdamW at lr 0.001 gives a small, consistent edge. Tuning traded a little recall for higher precision and tighter boxes — further gains now come from data, not hyperparameters.



Strong, honest results on held-out test

Inference-only on the frozen checkpoint over a strictly held-out split (371 images / 445 boxes, zero leakage). Smoking clearly outperforms drinking, and the failure mode is the safe one.

0.85

Precision (all)

0.80

Recall (all)

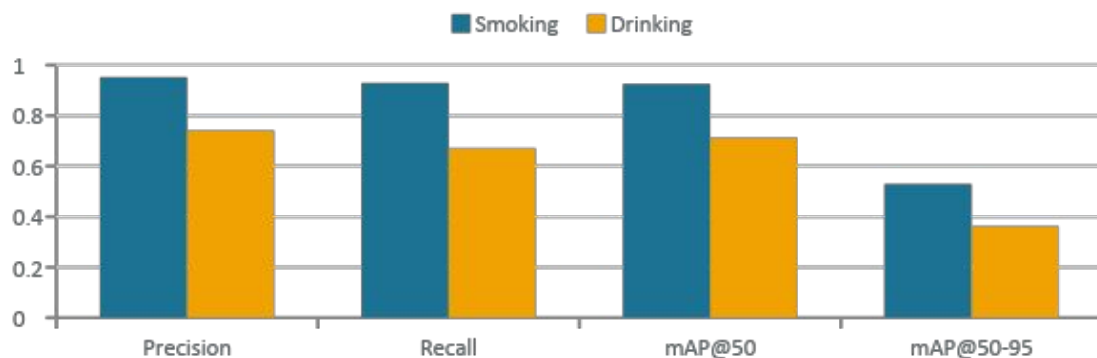
0.82

mAP@50 (all)

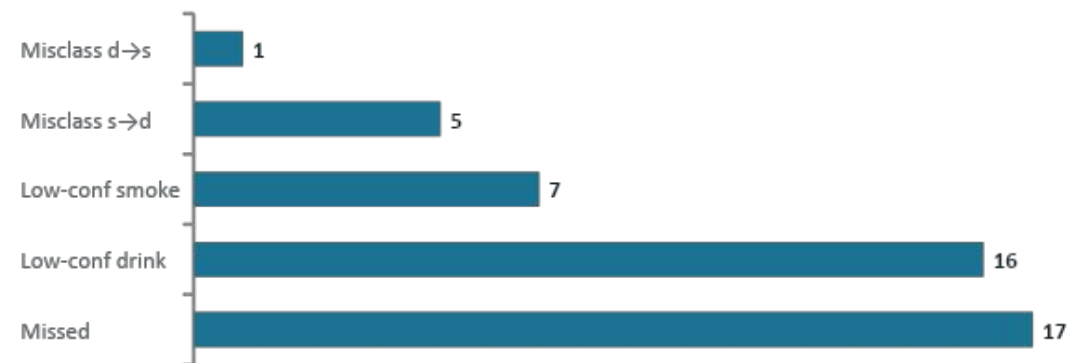
0.45

mAP@50-95 (all)

Per-class metrics — smoking vs drinking



Failure breakdown by error type (46 total)



The findings: Drinking is the weaker class — recall 0.67 vs 0.93 for smoking (~0.26 gap), the primary improvement target. But only 6 of 46 failures are true cross-class confusion; the rest are missed or low-confidence boxes — the model fails to fire rather than asserting the wrong class, the safe direction for a safety system. Validation (0.442) and test (0.447) mAP@50-95 match to within 0.005 → genuine generalization, no leakage. Image-level top-1 accuracy is ~88% (325 / 371), and the model runs real-time at ~145 FPS on a T4.

Five Signals Fused Into One Live Score

From one model's single "fatigue: yes / no" label → five models fused into one continuous 0–100 score, updated every second from the stream.

$$R_i = \text{severity} \times \text{confidence}$$

SEVERITY WEIGHTS (0–10)

 **Video Fatigue** Medium 6 · High 10

 **Landmark** Drowsy 7 · Yawning 7 · Talking 4

 **Driver Activity** Texting 9 · Phone 8 · Other 6 · Turn 4

 **Seat Belt & Phone** Phone 9 · Phone+Belt 9 · None 8

 **Smoking / Drinking** Drinking 10 · Both 10 · Smoking 5

THE SCORING PIPELINE

1 Event risk
 $R_i = \text{severity} \times \text{confidence}$

2 Safe = 0
non-risky labels add nothing

3 Total risk
 $R_{\text{total}} = \sum R_i$

4 Exponential fuse
 $100 \times (1 - e^{(-k \cdot R_{\text{total}})})$, k = hyperparameter

5 Smooth
3-second moving average to reduce flicker

6 Classify
Low → Moderate → High → Critical

The engine consumes only each module's label + confidence — never the internal network — so any model can be swapped or retrained without touching the math.

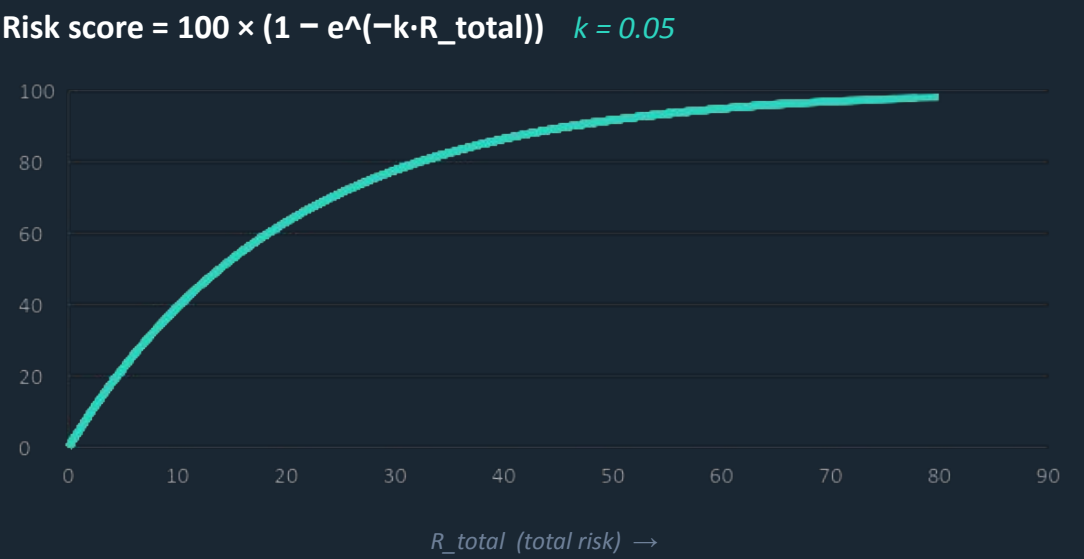
Danger × Certainty — a Worked Example

MODULE	PREDICTION	SEV	CONF	R _i
Landmark	Drowsy	7	0.90	6.3
Driver Activity	texting_phone	9	0.80	7.2
Seat Belt	No Detection	8	0.75	6.0
Video Fatigue	Low Risk (safe)	0	—	0
Smoking	none (safe)	0	—	0

R_{total} = 19.5

score = 100 × (1 − e^{^(−0.05 × 19.5)})

= 62.29 → HIGH RISK



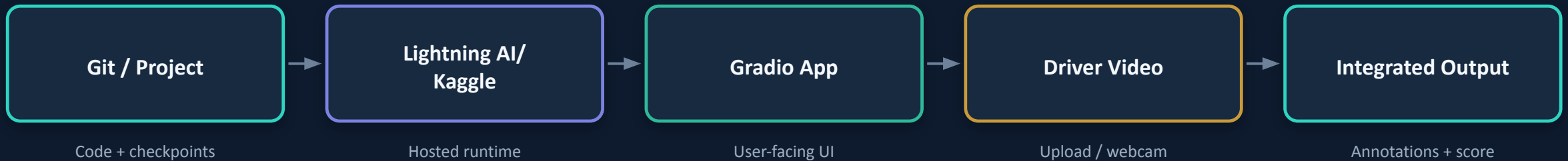
- Low Risk0 – 25
- Moderate Risk25 – 50
- High Risk50 – 75
- Critical Risk75 – 100

Displayed score = mean of raw scores over the last 3.0 s — killing per-frame flicker before the level is chosen.

Deployment — Lightning AI/Kaggle + Gradio

Packaging the integrated application for hosted inference.

FINAL DEPLOYMENT FLOW



DEPLOYMENT ENGINEERING

- Thin application wrapper + centralized inference core.
 - Model caching / lazy loading and frame skipping reduce startup and runtime resource pressure.
 - requirements.txt and README.md document the reproducible runtime.
 - Gradio exposes the same integrated inference pipeline rather than separate module demos.
-
- Lightning AI: `python app.py + Gradio share=True` → public `gradio.live` URL
 - Kaggle fallback: `python run_kaggle_gradio_fallback.py` on GPU with `share=True` → same live demo link

Thank You

Time for Live Demo

AI-Powered Driver Wellness System — five models, one fused wellness score

We'll now switch to the integrated Gradio demo / Risk Fusion Engine in action